



Experiment No 05

Implementation of Classes and Objects

Objective

An introduction with Classes, Objects, Constructors and Destructors

Software Tools

Visual Studio/ Dev++

C++ What is OOP?

OOP stands for Object-Oriented Programming.

Procedural programming is about writing procedures or functions that perform operations on the data, while object-oriented programming is about creating objects that contain both data and functions.

Object-oriented programming has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C++ code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

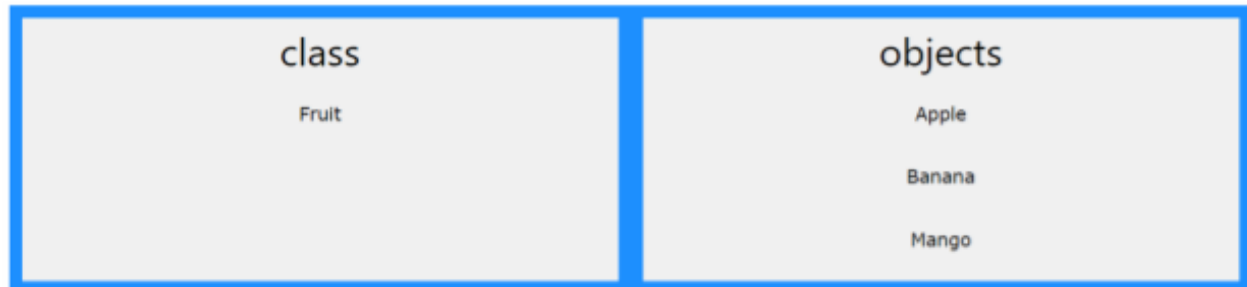
Tip: The "Don't Repeat Yourself" (DRY) principle is about reducing the repetition of code. You should extract out the codes that are common for the application, and place them at a single place and reuse them instead of repeating it.



C++ What are Classes and Objects?

Classes and objects are the two main aspects of object-oriented programming.

Look at the following illustration to see the difference between class and objects:



Another example:

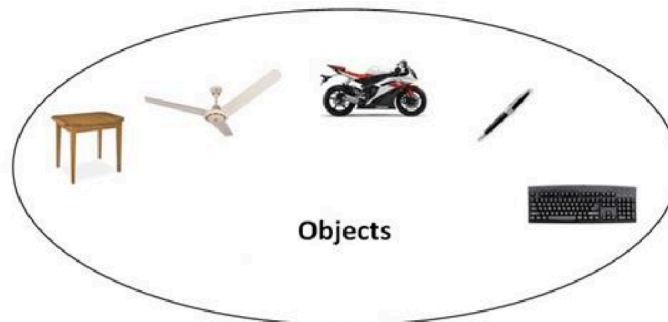


So, a class is a template for objects, and an object is an instance of a class.

When the individual objects are created, they inherit all the variables and functions from the class.

OPPs (Object Oriented Programming System)

Object means a real word entity such as pen, chair, table etc. **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies the software development and maintenance by providing some concepts:



Object

Any entity that has state and behavior is known as an object. For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.

Class

Collection of objects is called class. It is a logical entity.

C++ Class

Before you create an object in C++, you need to define a class.

A class is a blueprint for the object. We can think of class as a sketch (prototype) of a house. It contains all the details about the floors, doors, windows etc. Based on these descriptions we build the house. House is the object.

As, many houses can be made from the same description, we can create many objects from a class.

How to define a class in C++?

A class is defined in C++ using keyword `class` followed by the name of class. The body of class is defined inside the curly brackets and terminated by a semicolon at the end.

```
class className {
```



```
// some data

// some functions

};
```

Example: Class in C++

```
1. class Test
2. {
3.     private:
4.         int data1;
5.         float data2;
6.
7.     public:
8.         void function1()
9.         { data1 = 2; }
10.
11.        float function2()
12.        {
13.            data2 = 3.5;
14.            return data2;
15.        }
16. };
```

Here, we defined a class named `Test`.

This class has two data members: `data1` and `data2` and two member functions: `function1()` and `function2()`.

Keywords: private and public

You may have noticed two keywords: `private` and `public` in the above example.

The `private` keyword makes data and functions private. Private data and functions can be accessed only from inside the same class.

The `public` keyword makes data and functions public. Public data and functions can be accessed out of the class.

Here, `data1` and `data2` are private members where as `function1()` and `function2()` are public members.



If you try to access private data from outside of the class, compiler throws error. This feature in OOP is known as data hiding.

C++ Objects

When class is defined, only the specification for the object is defined; no memory or storage is allocated.

To use the data and access functions defined in the class, you need to create objects.

Syntax to Define Object in C++

```
className objectVariableName;
```



You can create objects of `Test` class (defined in above example) as follows:

```
1.
2. class Test
3. {
4.     private:
5.         int data1;
6.         float data2;
7.
8.     public:
9.         void function1()
10.        { data1 = 2; }
11.
12.        float function2()
13.        {
14.            data2 = 3.5;
15.            return data2;
16.        }
17. };
18.
19. int main()
20. {
21.     Test o1, o2;
22. }
```

Here, two objects `o1` and `o2` of `Test` class are created.
In the above class `Test`, `data1` and `data2` are data members
and `function1()` and `function2()` are member functions.

How to access data member and member function in C++?

You can access the data members and member functions by using a `.` (dot) operator.
For example,

```
o2.function1();
```

This will call the `function1()` function inside the `Test` class for objects `o2`.

Similarly, the data member can be accessed as:



```
o1.data2 = 5.5;
```

It is important to note that, the private members can be accessed only from inside the class.

So, you can use `o2.function1();` from any function or class in the above example. However, the code `o1.data2 = 5.5;` should always be inside the class `Test`.

Example: Object and Class in C++ Programming

```
1. // Program to illustrate the working of objects and class in C++ Programming
2. #include <iostream>
3. using namespace std;
4.
5. class Test
6. {   private:
7.     int data1;
8.     float data2;
9.
10.
11.
12. public:
13.
14.     void insertIntegerData(int d)
15.     {
16.         data1 = d;
17.         cout << "Number: " << data1;
18.     }
19.
20.     float insertFloatData()
21.     {
22.         cout << "\nEnter data: ";
23.         cin >> data2;
24.         return data2;
25.     }
26. };
27.
28. int main()
29. {
30.     Test o1, o2;
31.     float secondDataOfObject2;
32.
```



```
33.  
34.             insertFloatData();  
35.     o1.insertIntegerData(12);  
36.     secondDataOfObject2 = o2.  
37.     cout << "You entered " << secondDataOfObject2;  
     return 0; }
```

Output

```
Number: 12  
Enter data: 23.3  
You entered 23.3
```

In this program, two data members `data1` and `data2` and two member functions `insertIntegerData()` and `insertFloatData()` are defined under `Test` class. Two objects `o1` and `o2` of the same class are declared.

The `insertIntegerData()` function is called for the `o1` object using:

```
o1.insertIntegerData(12);
```

This sets the value of `data1` for object `o1` to 12.

Then, the `insertFloatData()` function for object `o2` is called and the return value from the function is stored in variable `secondDataOfObject2` using:

```
secondDataOfObject2 = o2.insertFloatData();
```

In this program, `data2` of `o1` and `data1` of `o2` are not used and contains garbage value.

Access Specifiers

By now, you are quite familiar with the `public` keyword that appears in all of our class examples:

Example

```
class MyClass { // The class  
    public:     // Access specifier
```




```
// class members goes here  
};
```

The `public` keyword is an **access specifier**. Access specifiers define how the members (attributes and methods) of a class can be accessed. In the example above, the members are `public` - which means that they can be accessed and modified from outside the code.

However, what if we want members to be private and hidden from the outside world?

In C++, there are three access specifiers:

- `public` - members are accessible from outside the class
- `private` - members cannot be accessed (or viewed) from outside the class
- `protected` - members cannot be accessed from outside the class, however, they can be accessed in inherited classes. You will learn more about Inheritance later.

In the following example, we demonstrate the differences between `public` and `private` members:

Example

```
class MyClass {  
    public:    // Public access specifier  
    int x;    // Public attribute    private:  
    // Private access specifier    int y;  
    // Private attribute  
};  
int main() {    MyClass myObj;    myObj.x  
= 25;    // Allowed (public)    myObj.y = 50;  
    // Not allowed (private)    return 0;  
}
```

If you try to access a private member, an error occurs:

```
error: y is private
```



Note: It is possible to access private members of a class using a public method inside the same class. We will see in the Encapsulation topic on how to do this.

Tip: It is considered good practice to declare your class attributes as private (as often as you can). This will reduce the possibility of yourself (or others) to mess up the code. This is also the main ingredient of the Encapsulation concept, which you will learn more about in the next chapter.

Note: By default, all members of a class are **private** if you don't specify an access specifier:

Example

```
class MyClass {  
    int x;    // Private attribute  
    int y;    // Private attribute  
};
```

Multiple Objects

You can create multiple objects of one class:

Example

```
// Create a Car class with some attributes  
class Car {  
public:  
    string brand;  
    string model;    int  
    year; };  
int main() {  
    // Create an object of Car  
    Car carObj1;    carObj1.brand  
    = "BMW";    carObj1.model =  
    "X5";    carObj1.year = 1999;  
  
    // Create another object of Car  
    Car carObj2;    carObj2.brand =
```



```
"Ford";    carObj2.model =  
"Mustang";    carObj2.year = 1969;  
  
    // Print attribute values  
    cout << carObj1.brand << " " << carObj1.model << " " <<  
carObj1.year << "\n";    cout << carObj2.brand << " " <<  
carObj2.model << " " << carObj2.year << "\n";    return 0;  
}
```

Class Methods

Methods are **functions** that belongs to the class.

There are two ways to define functions that belongs to a class:

- Inside class definition
- Outside class definition

In the following example, we define a function inside the class, and we name it "myMethod".

Note: You access methods just like you access attributes; by creating an object of the class and by using the dot syntax (.):

Inside Example

```
class MyClass {           // The class  
    public:               // Access specifier  
  
    void myMethod() {     // Method/function defined inside the class  
cout << "Hello World!";  
    }  
};  
  
int main() {
```



```
MyClass myObj;    // Create an object of MyClass
myObj.myMethod(); // Call the method

return 0;
}
```

To define a function outside the class definition, you have to declare it inside the class and then define it outside of the class. This is done by specifying the name of the class, followed the scope resolution `::` operator, followed by the name of the function:

Outside Example

```
class MyClass {           // The class
public:                  // Access specifier    void
myMethod();             // Method/function declaration
};

// Method/function definition outside the class
void MyClass::myMethod()
{
    cout << "Hello World!";
}

int main() {
    MyClass myObj;        // Create an object of MyClass
myObj.myMethod();        // Call the method    return 0;
}
```

Parameters

You can also add parameters:



Example

```
#include <iostream>
using namespace std;

class Car {
public:
    int speed(int maxSpeed);
};

int Car::speed(int maxSpeed) {
return maxSpeed;
}

int main()
{
    Car myObj; // Create an object of Car    cout <<
myObj.speed(200); // Call the method with an argument    return
0;
}
```

Task No 1:

- Write a program in C++ based on the following requirements: An EMPLOYEE class is to contain the following data members and member functions: Data members: EmployeeNumber (an integer), EmployeeName (a string of characters), BasicSalary (an integer), All Allowances (an integer), IT (an integer), NetSalary (an integer).
- Member functions: to read the data of an employee, to calculate Net Salary and to print the values of all the data members. (AllAllowances = 123% of Basic; Income Tax (IT) = 30% of the gross salary (= basic Salary_AllAllowance); Net Salary = Basic Salary + All Allowances – IT)

Task No 2:

- Write a program in C++ based on the following requirements: Create a class Country. Country is identified by its name, location on earth (which means the longitude & latitude values), area, population & capital.



- All the data members should be private. User should be able to set the value of all of its parameters either at the time of instance creation or later on be able to individually change the values of all these identifiers whenever user want to.
- Calling member function `DisplayAllInfo()` should display all the information pertaining to a country instance. User should also be able to individually call functions such as `DisplayName()`, `DisplayCapital` etc.
- To test: Ask the user to enter 3 countries and all the related info. Display the info once entered.

Lab Performance Evaluation:

Read and practice the manual. Performance is evaluated at the end of lab based on successfully running and understanding of examples, tasks and viva using defined rubrics.

Important note: Lab Report must be according to the Lab Report Format available on Google Classroom and must be submitted on time. Two similar reports will get zero marks. So, don't try to copy your fellow reports.